AUDIT REPORT · AUD-20261103-297B

paiement-api — microservice de virements SEPA — RTS 2024/1774 · Politique interne PDS-2 (Banque Kerlann, fictive)

Banque Kerlann (fictive) · paiement-api — microservice de virements SEPA

Lead auditor: **Attest CI**

Owner: **DSI — équipe de développement Paiements**

Date: **2026-11-03 23:00**

Run: **20261103-230028-297b**

1. Executive summary

OPINION AT TESTING

Unsatisfactory

7 controls tested	**7** findings	**0** findings closed and verified	**7** open findings
0 human approvals	**18** sensitive data items masked in evidence	**3 min 10 s** audit duration	**$0.328 *** measured AI cost

Critical: 1 · High: 3 · Medium: 3 · Low: 0

Key messages

- 7 findings across 7 controls: 1 critical, 3 high, 3 medium.
- 7 findings remain open, with an action plan, an owner and a due date.

2. Scope and method

Engagement	AUD-20261103-297B
Entity	Banque Kerlann (fictive)
System	paiement-api — microservice de virements SEPA
System criticality	Supporte des fonctions critiques ou importantes (DORA)
Environment	Pré-production de démonstration
Owner	DSI — équipe de développement Paiements
Lead auditor	Attest CI
Date	2026-11-03 23:00
Framework	RTS 2024/1774 — article 16 : acquisition, développement et maintenance des systèmes TIC · Politique interne PDS-2 (Banque Kerlann, fictive) —

	développement sécurisé
Audited target	paiement-api code repository (SEPA transfer microservice)
Work programme	reviewed by an auditor (Attest CI) · runs/studio/20261103-230028-297b/catalogue.json

In scope

- Code source, configuration et chaîne CI du dépôt
- Jeux de données de test versionnés

Out of scope

- Tests dynamiques (DAST) et tests d'intrusion
- Environnements d'exécution et de production
- Historique git

Method

1. Controls are derived from the framework (DORA, RTS 2024/1774, internal policy); severity is the one set by the policy.
2. Each test is a stored Python script, statically checked before execution (read-only) and run with a read-only service account.
3. The verdict comes from the script, never from the model. A screenshot is reviewed by Muse Spark to corroborate; on disagreement the control is flagged for review.
4. SAM 3.1 delineates the proving rows and the personal data, which is masked before distribution.
5. Every fix goes through a change request approved by the system owner, applied with a dedicated account, logged, then verified by the same test (DORA 9(4)(e)).
6. Every event and piece of evidence is sealed in a hash-chained audit trail (SHA-256).

Limitations

- Tests cover the configuration reported by the system, not the actual strength of authentication factors.
- AI-drafted scripts and catalogue must be reviewed by an auditor before being considered validated.
- Fictitious entity and data: demonstration.

3. Findings overview

REF.	RULE	TITLE	SEVERITY	TESTED	AFTER	STATUS	DUE DATE
F-01	PDS-2.1	No hardcoded secrets in code or versioned configuration	Critical	Non-compliant		Open	2026-12-03
F-02	PDS-2.2	TLS certificate verification never disabled	High	Non-compliant		Open	2027-02-01
F-03	PDS-2.3	Approved password hashing and ban of MD5/SHA-1	High	Non-compliant		Open	2027-02-01
F-04	PDS-2.6	Synthetic test data only	High	Non-compliant		Open	2027-02-01
F-05	PDS-2.4	Static security analysis in CI on each change	Medium	Non-compliant		Open	2027-05-02
F-06	PDS-2.5	Exact dependency pinning and CI audit	Medium	Non-compliant		Open	2027-05-02

F-07	PDS-2.7	Logging of authentication successes and failures	Medium	Non-compliant	Open	2027-05-02

4. Detailed findings and controls

F-01 PDS-2.1 No hardcoded secrets in code or versioned configuration — Critical — Non-compliant — Open

CRITERIA

Verify no passwords, keys or tokens are hardcoded in clear text and secrets are sourced from vault or environment variables.

PDS-2.1 · RTS 7(2)

Threshold source: PDS-2.1

> *« Aucun secret (mot de passe, clé, jeton) n'est écrit en clair dans le code source ni dans la configuration versionnée : les secrets viennent d'un coffre ou de variables d'environnement. »*

CONDITION

Observed: `{"match_count": 4, "checked_files": ["app/auth.py", "app/config.py"], "hardcoded": [{"path": "app/config.py", "line": 6, "text": "DB_PASSWORD = \"Az••••\" # TO…`

Expected: `Zero hardcoded clear-text secrets; all secret usages reference vault or environment variables.`

Searched 4 assignments across 2 files and found 2 hardcoded clear-text secrets without environment or vault reference.

RISK

Hardcoded secrets allow credential theft and unauthorized access to the SEPA transfer service.

RECOMMENDATION

Remove hardcoded secrets and source them from vault or environment variables.

Due date: 2026-12-03 · Owner: DSI — équipe de développement Paiements

Test procedure: GET /api/repo/paiement-api/tree to list files, GET /api/repo/paiement-api/search?q=(?i)(password|passwd|secret|api[_-]?key|token)\s*[:=] to find secret assignments, then GET /api/repo/paiement-api/file?path=<file> for matches to confirm values come from os.environ, os.getenv or vault client and no clear-text value is present.



At testing — The screenshot corroborates the test. Screenshot shows two hardcoded clear-text secrets in app/config.py matching the automated finding. · 3/3 sensitive data items masked (2 spotted by the AI, the rest by the deterministic safety net)

2026-11-03 23:02:18 · SHA-256 352334f02e9c7569…

F-02 PDS-2.2 TLS certificate verification never disabled — High — Non-compliant — Open

CRITERIA

Verify outgoing network calls verify TLS certificates and verify=False is absent.

PDS-2.2 · DORA 9(3)(a) ; RTS 6(2)(a)

Threshold source: PDS-2.2

> *« Tout appel réseau sortant vérifie le certificat TLS du serveur : la vérification n'est jamais désactivée (`verify=False` interdit). »*

CONDITION

Observed: `{"verify_False_matches": 1, "outgoing_refs": 1, "files_checked": 2, "confirmed_bypass": [{"path": "app/payments.py", "line": 18, "text": "response = requests.po…`

Expected: `Zero occurrences of verify=False or equivalent TLS verification bypass.`

Found 1 TLS verification bypass occurrence(s) with verify=False or equivalent in checked files.

RISK

Disabled TLS verification exposes SEPA transfers to interception and man-in-the-middle attacks.

RECOMMENDATION

Enable TLS certificate verification on all outgoing calls and remove any verify=False.

Due date: 2027-02-01 · Owner: DSI — équipe de développement Paiements

Test procedure: GET /api/repo/paiement-api/tree to list files, GET /api/repo/paiement-api/search?q=verify\s*=\s*False to find disabled verification, GET /api/repo/paiement-api/search?q=requests\.|urllib|httpx|http\.client to enumerate outgoing calls, then GET /api/repo/paiement-api/file?path=<file> to confirm TLS verification is enabled.



At testing — The screenshot corroborates the test. Screenshot shows line 18 in app/payments.py using verify=False, confirming disabled TLS verification.

2026-11-03 23:01:52 · SHA-256 7102af261c98f2e6…

F-03 PDS-2.3 Approved password hashing and ban of MD5/SHA-1

High · Non-compliant · Open

CRITERIA

Verify password hashing uses only approved functions and MD5/SHA-1 are not used for security.

PDS-2.3 · RTS 6(3) et 6(4)

Threshold source: PDS-2.3

> *« Les mots de passe sont hachés avec argon2, bcrypt, scrypt ou PBKDF2. MD5 et SHA-1 sont interdits pour tout usage de sécurité. »*

CONDITION

Observed: `{"approved_matches": [], "banned_matches": [{"line": 8, "path": "app/auth.py", "text": "return hashlib.md5(password.encode()).hexdigest()"}] "security_banned_u…`

Expected: `Password hashing uses argon2, bcrypt, scrypt or PBKDF2 only; zero MD5 or SHA-1 uses for security.`

Found 1 MD5/SHA-1 security use(s) in app/auth.py with 0 approved hashing use(s)

RISK

Weak hashes allow password recovery and authentication bypass.

RECOMMENDATION

Replace MD5/SHA-1 with argon2, bcrypt, scrypt or PBKDF2 for all password hashing.

Due date: 2027-02-01 · Owner: DSI — équipe de développement Paiements

Test procedure: GET /api/repo/paiement-api/tree to list files, GET /api/repo/paiement-api/search?q=(argon2|bcrypt|scrypt|pbkdf2) and GET /api/repo/paiement-api/search?q=(md5|sha1|hashlib\.(md5|sha1)) to inventory hashing uses, then GET /api/repo/paiement-api/file?path=<file> to confirm password hashing uses an allowed algorithm and MD5/SHA-1 are absent from security contexts.



At testing — The screenshot corroborates the test. The screenshot shows app/auth.py line 8 using hashlib.md5 inside hash_password with no approved hashing function visible.
`2026-11-03 23:01:59 · SHA-256 9b670c1eafd915cb…`

F-04 PDS-2.6 Synthetic test data only

High · Non-compliant · Open

CRITERIA

Verify test data is declared synthetic and no production export is versioned.

PDS-2.6 · RTS 16(5)(a)

Threshold source: PDS-2.6

> *« Les données de test sont synthétiques et déclarées comme telles (champ `"synthetic": true`) ; aucun export de production n'est versionné. »*

CONDITION

Observed: `{"test_files": ["tests/fixtures/clients_export_prod.json", "tests/fixtures/operateurs_synthetiques.json" "tests/test_payments.py"], "data_files": ["tests/fixtu…`

Expected: `Test data files contain "synthetic": true; zero versioned production exports.`

3 tests/ files checked, 1 contain "synthetic": true and 1 contain production export indicators.

RISK

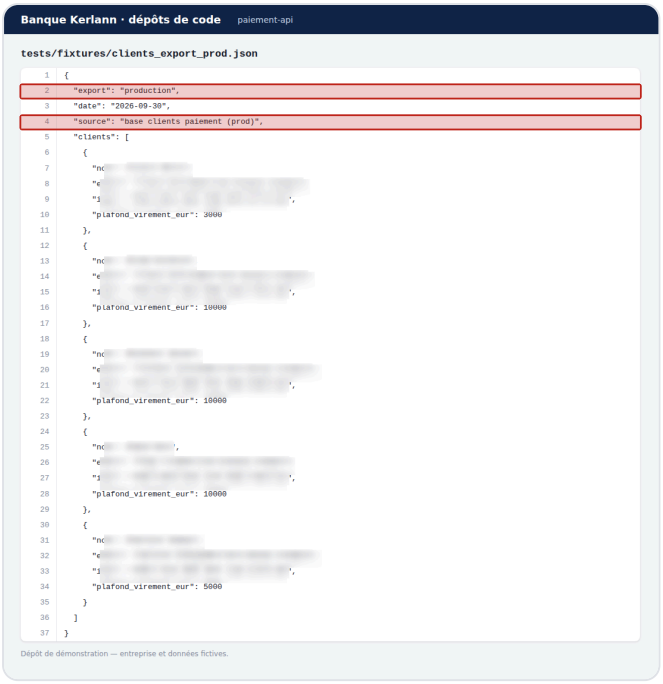
Production data in test fixtures causes data leakage and non-compliance.

RECOMMENDATION

Replace test data with synthetic data marked "synthetic": true and remove production exports.

Due date: 2027-02-01 · Owner: DSI — équipe de développement Paiements

Test procedure: GET /api/repo/paiement-api/tree to list tests/ fixtures, GET /api/repo/paiement-api/search?q=synthetic to find synthetic declarations, GET /api/repo/paiement-api/file?path=<fixture> to confirm presence of "synthetic": true and absence of production data exports.



At testing — The screenshot corroborates the test. The file declares a production export from a production payment database with no synthetic flag. · 15/15 sensitive data items masked

2026-11-03 23:03:06 · SHA-256 b3d27f0def9dd118…

F-05 PDS-2.4 Static security analysis in CI on each change

Medium | Non-compliant | Open

CRITERIA

Verify CI runs bandit, semgrep or CodeQL on each modification.

PDS-2.4 · RTS 16(3)

Threshold source: PDS-2.4

> *« La chaîne d'intégration continue exécute une analyse statique de sécurité (bandit, semgrep ou CodeQL) à chaque modification. »*

CONDITION

Observed: `{"workflow_files": [".github/workflows/ci.yml"], "matches": [], "checks": {".github/workflows/ci.yml": {"has_analyser": false, "triggered": true, "tool": null}}…`

Expected: `CI workflow contains bandit or semgrep or CodeQL execution triggered on each modification.`

No workflow in .github/workflows/ executes bandit, semgrep or CodeQL.

RISK

Missing static analysis leaves code vulnerabilities undetected before deployment.

RECOMMENDATION

Add bandit, semgrep or CodeQL to the CI pipeline on every change.

Due date: 2027-05-02 · Owner: DSI — équipe de développement Paiements

Test procedure: GET /api/repo/paiement-api/tree to list .github/workflows files, GET /api/repo/paiement-api/search?q=(bandit|semgrep|codeql) to find analyser invocations, then GET /api/repo/paiement-api/file?path=.github/workflows/<file> to confirm the job runs on push/pull_request for every change.



At testing — The screenshot corroborates the test. The ci.yml workflow only runs checkout, setup-python, install and pytest with no bandit, semgrep or CodeQL step.
2026-11-03 23:02:08 · SHA-256 c167845e5c6b32ae…

F-06 PDS-2.5 Exact dependency pinning and CI audit

Medium | Non-compliant | Open

CRITERIA

Verify dependencies are pinned with == and audited in CI with pip-audit or equivalent.
PDS-2.5 · RTS 16(4) et 16(8)
Threshold source: PDS-2.5

> *« Les dépendances sont épinglées à une version exacte (`paquet==version`) et auditées en intégration continue (pip-audit ou équivalent). »*

CONDITION

Observed: `{"requirements_lines": ["flask", "requests>=2.0", "pyjwt==2.8.0"], "considered": 3, "pinned_count": 1, "unpinned": ["flask", "requests>=2.0"], "workflows": [".g…`
Expected: `All requirements use paquet==version exact pinning; CI runs pip-audit or equivalent.`
requirements.txt shows 1/3 lines pinned with 2 unpinned and CI audit was not confirmed in any workflow file.

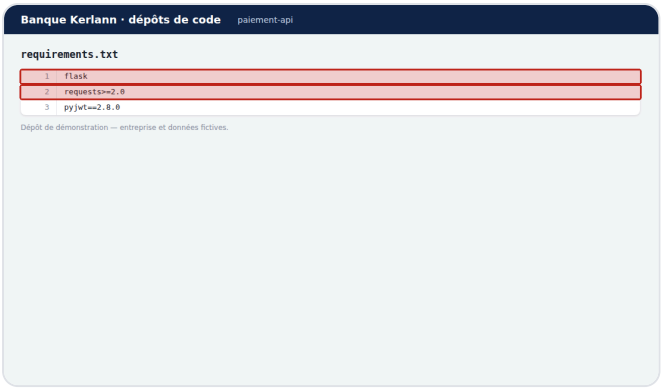
RISK

Unpinned or unaudited dependencies allow vulnerable packages into production.

RECOMMENDATION

Pin all dependencies with == and add pip-audit or equivalent to CI.
Due date: 2027-05-02 · Owner: DSI — équipe de développement Paiements

Test procedure: GET /api/repo/paiement-api/tree to locate requirements.txt and .github/workflows files, GET /api/repo/paiement-api/file?path=requirements.txt to verify each line uses == exact pinning, GET /api/repo/paiement-api/search?q=(pip-audit|pip_audit|safety|dependabot) to verify audit in CI, then GET /api/repo/paiement-api/file?path=.github/workflows/<file> to confirm the audit step.



At testing — The screenshot corroborates the test. Screenshot shows flask without version and requests with >= constraint, matching the two unpinned dependencies reported.
2026-11-03 23:03:01 · SHA-256 94071ff0cc3ec301…

F-07 PDS-2.7 Logging of authentication successes and failures

Medium | Non-compliant | Open

CRITERIA

Verify the application logs both successful and failed authentication events.
PDS-2.7 · RTS 12(2)(c)(i)
Threshold source: PDS-2.7

> *« Les réussites et les échecs d'authentification sont journalisés par l'application. »*

CONDITION

Observed: `{"auth_files": ["app/auth.py"], "candidate_files_checked": ["app/auth.py"], "success_logs": [], "failure_logs": [], "all_repo_files_count": 11}`
Expected: `Code contains log statements on both authentication success and authentication failure paths.`
Checked 1 auth file(s) found 0 success log(s) and 0 failure log(s), missing both success and failure auth logs.

RISK

Missing auth logs prevent detection of brute-force and account takeover.

RECOMMENDATION

Add logging for both successful and failed authentication attempts.
Due date: 2027-05-02 · Owner: DSI — équipe de développement Paiements

Test procedure: GET /api/repo/paiement-api/tree to list app/ files, GET /api/repo/paiement-api/search?q=(login|auth|authenticate) and GET /api/repo/paiement-api/search?q=(log|logger|logging) to find auth logging, then GET /api/repo/paiement-api/file?path=<file> to confirm both success and failure paths emit a log record.



At testing — The screenshot corroborates the test. The login function returns on both failure and password-check paths without any logging call.

2026-11-03 23:03:38 · SHA-256 a9db72607ca95e4a…

Appendices

A. Traceability matrix

REQUIREMENT	RULE	CONTROL	SCRIPT (SHA-256)	RESULT
RTS 7(2)	PDS-2.1	C1 · No hardcoded secrets in code or versioned configuration	e8e16edce8e98b01 · written by AI for this engagement	Non-compliant
DORA 9(3)(a) ; RTS 6(2)(a)	PDS-2.2	C2 · TLS certificate verification never disabled	9cf044c19636d28a · written by AI for this engagement	Non-compliant
RTS 6(3) et 6(4)	PDS-2.3	C3 · Approved password hashing and ban of MD5/SHA-1	87295e95fc0a4eef · written by AI for this engagement	Non-compliant
RTS 16(3)	PDS-2.4	C4 · Static security analysis in CI on each change	e38a3e4fefc4ad4c · written by AI for this engagement	Non-compliant
RTS 16(4) et 16(8)	PDS-2.5	C5 · Exact dependency pinning and CI audit	701126d1db10733f · written by AI for this engagement	Non-compliant
RTS 16(5)(a)	PDS-2.6	C6 · Synthetic test data only	40ab2b4015200921 · written by AI for this engagement	Non-compliant
RTS 12(2)(c)(i)	PDS-2.7	C7 · Logging of authentication successes and failures	265038f0761e0aa4 · written by AI for this engagement	Non-compliant

B. Evidence register

FILE			SHA-256
evidence/C1_before.png	C1 · At testing	2026-11-03 23:02:18	352334f02e9c75690170d30fe42232e2c9c497519cccd991e4442b79f60c3d70
evidence/C2_before.png	C2 · At testing	2026-11-03 23:01:52	7102af261c98f2e6311f5cb0b2f8280dd8e4806b305a0b073854bfa88cf5a517
evidence/C3_before.png	C3 · At testing	2026-11-03 23:01:59	9b670c1eafd915cb6df201edf42dc67d9cd479c769ad7b5f49bc574e294bd891
evidence/C4_before.png	C4 · At testing	2026-11-03 23:02:08	c167845e5c6b32ae92f6580b3d69cc17661204d9320897bd1ee34c41a347a0d1
evidence/C5_before.png	C5 · At testing	2026-11-03 23:03:01	94071ff0cc3ec3013b5c4620d989ef9e69def04847069f8c7a03fba625a6574f
evidence/C6_before.png	C6 · At testing	2026-11-03 23:03:06	b3d27f0def9dd1184fc6b121ec246c5b760b57bd6eb2ef7d5c35b544409ed68e
evidence/C7_before.png	C7 · At testing	2026-11-03 23:03:38	a9db72607ca95e4a77d78480e6b47d918a658f43d6b9d2e0c0c4117d021ac7a9

C. AI usage, costs and human oversight

MODEL	CALLS	INPUT TOKENS	OUTPUT TOKENS	IMAGES	COST (USD)

muse-spark-1.3	15	20,822	71,151	0	0.3284
sam-3.1	6	0	0	6	price not published
Total					**0.3284**

Sensitive data (personal data and secrets) in evidence: 17/18 spotted by Muse Spark + SAM, 18/18 masked in the end thanks to the deterministic safety net (page text).

* sam-3.1: price not published (pricing.toml)

Safeguards

- Scripts: allow-listed imports, no write requests (static analysis before execution).
- Read-only test account: any write would be refused by the system (defence in depth).
- Fixes: allow-listed route and parameters, human approval, separate change account.
- Segregation of duties: the approver cannot be the auditor.

D. Integrity

Audit-trail fingerprint when the report was issued. Verify with `python -m attest verify <folder>`.

`e5fcc3596f82cd98d7f4fb9818ab685eb59c830aa54f714862bfce80a967fd30`

F. Framework coverage

20 requirements read: 11 tested automatically (7 controls; 4 regulation clauses covered through the policy), 9 left to document review, 0 to interviews, 0 rejected for lack of an exact quote. Attest only covers the first group.

RÉF.	TEXT (VERIFIED QUOTE)	TESTABILITY	CONTROL	
RTS 16(1)	« financial entities shall develop, document and implement a policy governing the acquisition, development, and maintenance of ICT systems. »	**document review**	0	Requires review of a governance policy document not visible via the read-only code repository API.
RTS 16(1)(a)	« identify security practices and methodologies relating to the acquisition, development, and maintenance of ICT systems; »	**document review**	0	Requires review of policy content not visible via the read-only code repository API.
RTS 16(1)(c)	« specify measures to mitigate the risk of unintentional alteration or intentional manipulation of the ICT systems during development, maintenance, and deployment. »	**document review**	0	Requires review of policy content not visible via the read-only code repository API.
RTS 16(2)	« financial entities shall develop, document, and implement an ICT systems' acquisition, development, and maintenance procedure for the testing and approval of all ICT systems prior to their use and after maintenance. »	**document review**	0	Requires review of a procedure document and approval records not visible via the read-only code repository API.
RTS 16(3)	« the procedure shall contain source code reviews covering both static and dynamic	**tested automatically**	→ PDS-2.4	Implementation is verifiable in CI configuration via the

	testing, including security testing for internet-exposed systems and applications. »			read-only repository API through covering policy rule PDS-2.4.
RTS 16(3)(a)	« identify and analyse vulnerabilities and anomalies in the source code; »	**document review**	0	Requires review of vulnerability analysis reports and action plans not visible via the read-only code repository API.
RTS 16(3)(b)	« adopt an action plan to address those vulnerabilities and anomalies; »	**document review**	0	Requires review of remediation action plans not visible via the read-only code repository API.
RTS 16(3)(c)	« monitor the implementation of that action plan. »	**document review**	0	Requires review of follow-up and monitoring evidence not visible via the read-only code repository API.
RTS 16(4)	« the procedure shall contain security testing of software packages no later than at the integration phase. »	**tested automatically**	→ PDS-2.5	Implementation is verifiable in CI and dependency configuration via the read-only repository API through covering policy rule PDS-2.5.
RTS 16(5)(a)	« non-production environments only store anonymised, pseudonymised, or randomised production data; »	**tested automatically**	→ PDS-2.6	Implementation is verifiable in test fixtures via the read-only repository API through covering policy rule PDS-2.6.
RTS 16(5)(b)	« financial entities are to protect the integrity and confidentiality of data in non-production environments. »	**document review**	0	Requires review of data protection measures for non-production environments not provable by code search alone.
RTS 16(7)	« the procedure shall include controls to protect the integrity of source code of ICT systems that are developed in-house or by an ICT third-party service provider. »	**document review**	0	Requires review of source-code integrity controls and repository settings not visible via the read-only file content API.
RTS 16(8)	« proprietary software and, where feasible, the source code provided by ICT third-party service providers, as well as source code coming from open-source projects, are to be analysed and tested in accordance with paragraph »	**tested automatically**	→ PDS-2.5	Implementation is verifiable in CI dependency audit configuration via the read-only repository API through covering policy rule PDS-2.5.

PDS-2.1	« Aucun secret (mot de passe, clé, jeton) n'est écrit en clair dans le code source ni dans la configuration versionnée : les secrets viennent d'un coffre ou de variables d'environnement. »	**tested automatically**	1	Verifiable by listing files and searching code for hardcoded secrets via the read-only repository API.
PDS-2.2	« Tout appel réseau sortant vérifie le certificat TLS du serveur : la vérification n'est jamais désactivée (`verify=False` interdit). »	**tested automatically**	1	Verifiable by searching code for disabled TLS verification via the read-only repository API.
PDS-2.3	« Les mots de passe sont hachés avec argon2, bcrypt, scrypt ou PBKDF2. MD5 et SHA-1 sont interdits pour tout usage de sécurité. »	**tested automatically**	1	Verifiable by searching code for hashing primitives via the read-only repository API.
PDS-2.4	« La chaîne d'intégration continue exécute une analyse statique de sécurité (bandit, semgrep ou CodeQL) à chaque modification. »	**tested automatically**	1	Verifiable by inspecting CI workflow files via the read-only repository API.
PDS-2.5	« Les dépendances sont épinglées à une version exacte (`paquet==version`) et auditées en intégration continue (pip-audit ou équivalent). »	**tested automatically**	1	Verifiable by inspecting requirements file and CI workflows via the read-only repository API.
PDS-2.6	« Les données de test sont synthétiques et déclarées comme telles (champ `"synthetic": true`) ; aucun export de production n'est versionné. »	**tested automatically**	1	Verifiable by inspecting test fixtures for synthetic marking via the read-only repository API.
PDS-2.7	« Les réussites et les échecs d'authentification sont journalisés par l'application. »	**tested automatically**	1	Verifiable by searching application code for authentication logging via the read-only repository API.